Name-Tejas Shekhar Kamble Roll No-CS22

Aim : To implement a **Logistic Regression** model to classify credit card transactions as **fraudulent or legitimate**.

# Objectives

1. To understand the mathematical foundations of **Logistic Regression** and the **Sigmoid Function**.
2. To understand and handle **class imbalance** in credit card fraud datasets.
3. To implement **Gradient Descent** for optimizing Logistic Regression parameters.
4. To evaluate the model using **Confusion Matrix, Precision, Recall, F1-Score, ROC Curve, and AUC**.
5. To understand the importance of **Recall and False Negative Rate** in fraud detection.

# Relevant Theory

## Logistic Regression

Logistic Regression is a supervised machine learning algorithm used for **binary classification**. In this experiment, it is used to classify credit card transactions into two classes:

- **0 $\rightarrow$ Legitimate transaction**
- **1 $\rightarrow$ Fraudulent transaction**

The model first calculates a linear combination of input features:

$$ z = w^T X + b $$
where:

- $w$ = weight vector
- $X$ = input feature vector
- $b$ = bias/intercept
- $z$ = linear predictor

---

## Sigmoid Function

The Sigmoid Function converts the value of $z$ into a probability between 0 and 1:

$$ \sigma(z) = \frac{1}{1 + e^{-z}} $$
The predicted probability that a transaction is fraudulent is:

$$ P(y=1|X) = \sigma(w^T X + b) $$
The predicted class can be obtained using a threshold:

$$ \hat{y} = \begin{cases} 1, & \text{if } P(y=1|X) \geq 0.5 \\ 0, & \text{if } P(y=1|X) < 0.5 \end{cases} $$

---

# Cost Function

Logistic Regression uses the **Binary Cross-Entropy Loss** or **Log Loss**.

The cost function is:

$$ J(w,b) = -\frac{1}{m} \sum_{i=1}^{m} \left[ y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i) \right] $$
where:

- $m$ = number of training examples
- $y_i$ = actual class label
- $\hat{y}_i$ = predicted probability

The objective of training is to find values of $w$ and $b$ that minimize the cost function.

---

# Gradient Descent

Gradient Descent is used to optimize the parameters of the Logistic Regression model.

The weights are updated using:

$$ w := w - \alpha \frac{\partial J}{\partial w} $$
The bias is updated using:

$$ b := b - \alpha \frac{\partial J}{\partial b} $$

where $\alpha$ is the learning rate.

For Logistic Regression, the gradient of the cost function with respect to the weights can be expressed as:

$$ \frac{\partial J}{\partial w} = \frac{1}{m}X^T(\hat{y}-y) $$
The gradient with respect to the bias is:

$$ \frac{\partial J}{\partial b} = \frac{1}{m} \sum_{i=1}^{m} (\hat{y}_i-y_i) $$
Therefore, the parameter updates become:

$$ w := w - \alpha \left[ \frac{1}{m}X^T(\hat{y}-y) \right] $$$$ b := b - \alpha \left[ \frac{1}{m} \sum_{i=1}^{m} (\hat{y}_i-y_i) \right] $$

---

# Class Imbalance

Credit card fraud datasets are generally highly imbalanced because legitimate transactions significantly outnumber fraudulent transactions.

For example, the dataset may contain a very large number of legitimate transactions and only a small number of fraudulent transactions.

A model that predicts most transactions as legitimate may achieve high accuracy but perform poorly at detecting fraud.

Therefore, techniques such as:

- Random Undersampling
- Oversampling
- Class Weighting

can be used to handle class imbalance.

---

# Evaluation Metrics

## Confusion Matrix

The confusion matrix consists of four components:

- **True Positive (TP):** Fraudulent transaction correctly identified as fraud.

- **True Negative (TN):** Legitimate transaction correctly identified as legitimate.
- **False Positive (FP):** Legitimate transaction incorrectly identified as fraud.
- **False Negative (FN):** Fraudulent transaction incorrectly identified as legitimate.

---

## Accuracy

Accuracy measures the proportion of correctly classified transactions:

$$ Accuracy = \frac{TP+TN} {TP+TN+FP+FN} $$
However, accuracy alone is not sufficient for highly imbalanced fraud detection datasets.

---

## Precision

Precision indicates the proportion of transactions predicted as fraudulent that are actually fraudulent:

$$ Precision = \frac{TP} {TP+FP} $$
A high Precision means that the model produces fewer false fraud alerts.

---

## Recall

Recall measures the proportion of actual fraudulent transactions correctly detected:

$$ Recall = \frac{TP} {TP+FN} $$
Recall is particularly important in fraud detection because a **False Negative** represents a fraudulent transaction that was not detected.

---

## F1-Score

F1-Score is the harmonic mean of Precision and Recall:

$$ F1 = 2 \times \frac{Precision \times Recall} {Precision + Recall} $$

---

# ROC Curve

The Receiver Operating Characteristic (ROC) curve represents the relationship between the **True Positive Rate (TPR)** and **False Positive Rate (FPR)** at different classification thresholds.

The True Positive Rate is:

$$ TPR = \frac{TP} {TP+FN} $$
The False Positive Rate is:

$$ FPR = \frac{FP} {FP+TN} $$
The ROC curve helps determine how well the model separates fraudulent transactions from legitimate transactions.

---

# *Area Under the Curve (AUC)*

The **Area Under the ROC Curve (AUC)** summarizes the performance of the classifier across different classification thresholds.

An AUC value close to:

$$ AUC = 1 $$
indicates excellent discrimination, while:

$$ AUC = 0.5 $$
indicates performance approximately equivalent to random classification.

---

## Dataset Information

The Credit Card Fraud Dataset contains transaction-related features such as transaction amount, distance from home, purchase-price-related ratios, and transaction characteristics.

The target variable is fraud:

0 → Legitimate transaction

1 → Fraudulent transaction

---

# Task 1: Environment Setup and Dataset Acquisition

Use dataset for this experiment Credit Card Fraud Dataset

Import the required Python libraries such as **NumPy, Pandas, Matplotlib, and Scikit-learn**.

Load the **Credit Card Fraud Dataset** into a Pandas DataFrame.

Display the following information:

- First five records of the dataset.
- Shape of the dataset.
- Column names.
- Data types of all columns.
- Descriptive statistics of numerical features.

Identify the **feature variables** and the **target variable** `fraud`.

The target variable is defined as:

$$ \text{Class Percentage} = \frac{\text{Number of Transactions in Class}}{\text{Total Number of Transactions}} \times 100 $$
fraud =

- 0 -> Legitimate transaction
- 1 -> Fraudulent transaction

The objective of this task is to understand the structure of the dataset and identify the input features and target variable required for building the Logistic Regression model.

Identify the feature variables and the target variable fraud.

In [2]:
```
# Import required libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confus

# Manually select and upload the dataset
```

```python
from google.colab import files

uploaded = files.upload()

# Get the uploaded file name automatically
file_name = list(uploaded.keys())[0]

# Load the dataset
df = pd.read_csv(file_name)

# Display first five records
print("First 5 Records:")
display(df.head())

# Display shape
print("\nShape of Dataset:")
print(df.shape)

# Display column names
print("\nColumn Names:")
print(df.columns.tolist())

# Display data types
print("\nData Types:")
print(df.dtypes)

# Display descriptive statistics
print("\nDescriptive Statistics:")
display(df.describe())

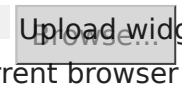
# Identify feature and target variables
X = df.drop(columns=['fraud'])
y = df['fraud']

print("\nFeature Variables:")
print(X.columns.tolist())

print("\nTarget Variable:")
print("fraud")

# Display class distribution
print("\nClass Distribution:")
print(y.value_counts())

# Calculate class percentage
print("\nClass Percentage:")
print(y.value_counts(normalize=True) * 100)
```




Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving card_transdata (1).csv to card_transdata (1).csv
First 5 Records:
```

| | distance_from_home | distance_from_last_transaction | ratio_to_median_purchase_price | repea |
|---|---|---|---|---|
| **0** | 57.877857 | 0.311140 | 1.945940 | |
| **1** | 10.829943 | 0.175592 | 1.294219 | |
| **2** | 5.091079 | 0.805153 | 0.427715 | |
| **3** | 2.247564 | 5.600044 | 0.362663 | |
| **4** | 44.190936 | 0.566486 | 2.222767 | |

```
Shape of Dataset:
(1000000, 8)

Column Names:
['distance_from_home', 'distance_from_last_transaction', 'ratio_to_media
n_purchase_price', 'repeat_retailer', 'used_chip', 'used_pin_number', 'on
line_order', 'fraud']

Data Types:
distance_from_home                float64
distance_from_last_transaction    float64
ratio_to_median_purchase_price    float64
repeat_retailer                   float64
used_chip                         float64
used_pin_number                   float64
online_order                      float64
fraud                             float64
dtype: object

Descriptive Statistics:
```

| | distance_from_home | distance_from_last_transaction | ratio_to_median_purchase_price | r |
|---|---|---|---|---|
| **count** | 1000000.000000 | 1000000.000000 | 1000000.000000 | 1 |
| **mean** | 26.628792 | 5.036519 | 1.824182 | |
| **std** | 65.390784 | 25.843093 | 2.799589 | |
| **min** | 0.004874 | 0.000118 | 0.004399 | |
| **25%** | 3.878008 | 0.296671 | 0.475673 | |
| **50%** | 9.967760 | 0.998650 | 0.997717 | |
| **75%** | 25.743985 | 3.355748 | 2.096370 | |
| **max** | 10632.723672 | 11851.104565 | 267.802942 | |

```
Feature Variables:
['distance_from_home', 'distance_from_last_transaction', 'ratio_to_media
n_purchase_price', 'repeat_retailer', 'used_chip', 'used_pin_number', 'on
line_order']

Target Variable:
fraud

Class Distribution:
fraud
0.0    912597
1.0     87403
Name: count, dtype: int64

Class Percentage:
fraud
0.0    91.2597
1.0     8.7403
Name: proportion, dtype: float64
```

# Task 2: Perform Exploratory Data Analysis

Perform **Exploratory Data Analysis (EDA)** to understand the structure, distribution, and characteristics of the Credit Card Fraud Dataset.

Calculate and display the following:

- Number of **legitimate transactions**.
- Number of **fraudulent transactions**.
- Percentage of legitimate transactions.
- Percentage of fraudulent transactions.
- Distribution of numerical features.

The percentage of each class can be calculated using: $$ \text{Class Percentage} = \frac{\text{Number of Transactions in Class}} {\text{Total Number of Transactions}} \times 100 $$

Plot the **class distribution** using a suitable visualization such as a bar chart.

Analyze the class distribution and explain the problem of **classification imbalance** in credit card fraud detection.

A dataset is considered highly imbalanced when the number of legitimate transactions is significantly greater than the number of fraudulent transactions. Therefore, accuracy alone may not be sufficient to evaluate the performance of the fraud detection model.

In [3]:
```
# Task 2: Exploratory Data Analysis

# Total number of transactions
total_transactions = len(df)

# Count legitimate and fraudulent transactions
legitimate = (df['fraud'] == 0).sum()
fraudulent = (df['fraud'] == 1).sum()

# Calculate percentages
legitimate_percentage = (legitimate / total_transactions) * 100
fraudulent_percentage = (fraudulent / total_transactions) * 100

# Display results
print("===== Class Distribution =====")
print("Total Transactions:", total_transactions)
print("Legitimate Transactions:", legitimate)
print("Fraudulent Transactions:", fraudulent)

print("\n===== Class Percentage =====")
print("Legitimate Transactions: {:.2f}%".format(legitimate_percentage))
print("Fraudulent Transactions: {:.2f}%".format(fraudulent_percentage))

# Distribution of numerical features
print("\n===== Distribution of Numerical Features =====")
display(df.describe())

# Plot class distribution
plt.figure(figsize=(6, 4))

df['fraud'].value_counts().sort_index().plot(
    kind='bar'
)

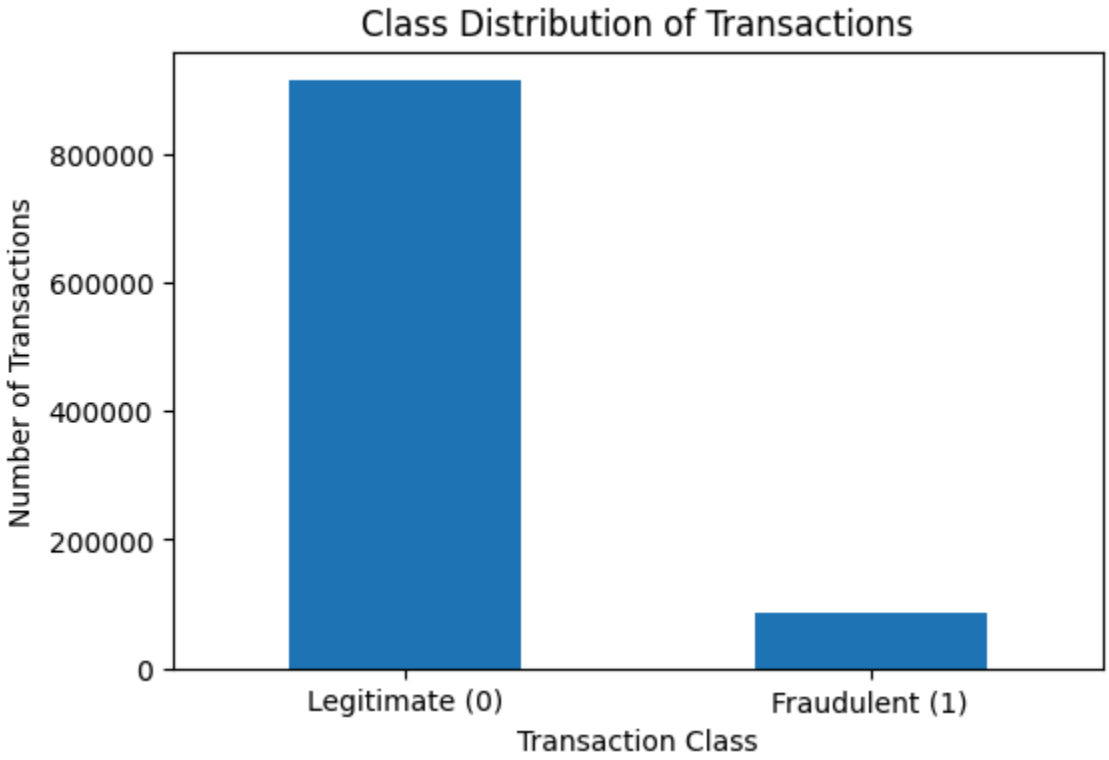
plt.title("Class Distribution of Transactions")
plt.xlabel("Transaction Class")
plt.ylabel("Number of Transactions")
plt.xticks([0, 1], ['Legitimate (0)', 'Fraudulent (1)'], rotation=0)
plt.show()
```

```
===== Class Distribution =====
Total Transactions: 1000000
Legitimate Transactions: 912597
Fraudulent Transactions: 87403

===== Class Percentage =====
Legitimate Transactions: 91.26%
Fraudulent Transactions: 8.74%

===== Distribution of Numerical Features =====
```

| | distance_from_home | distance_from_last_transaction | ratio_to_median_purchase_price | r |
|---|---|---|---|---|
| **count** | 1000000.000000 | 1000000.000000 | 1000000.000000 | 1 |
| **mean** | 26.628792 | 5.036519 | 1.824182 | |
| **std** | 65.390784 | 25.843093 | 2.799589 | |
| **min** | 0.004874 | 0.000118 | 0.004399 | |
| **25%** | 3.878008 | 0.296671 | 0.475673 | |
| **50%** | 9.967760 | 0.998650 | 0.997717 | |
| **75%** | 25.743985 | 3.355748 | 2.096370 | |
| **max** | 10632.723672 | 11851.104565 | 267.802942 | |




# Task 3: Data Preprocessing

Check the dataset for the following:

- Missing values.
- Duplicate records.
- Categorical variables.
- Numerical variables.

Identify the number of missing values in each column and handle them using suitable techniques such as **removal or imputation**.

Identify categorical variables and convert them into numerical form using appropriate **encoding techniques**.

For example, categorical variables can be converted into numerical values using **One-Hot Encoding**.

Also, identify the numerical features that will be used for building the Logistic Regression model.

The objective of this task is to prepare a clean and suitable dataset for further processing, feature scaling, model training, and evaluation.

In [4]:
```
# ============================================================
# TASK 3: DATA PREPROCESSING
# Credit Card Fraud Dataset
# ============================================================

import numpy as np
import pandas as pd

# ------------------------------------------------------------
# STEP 1: Check missing values
# ------------------------------------------------------------

print("========== MISSING VALUES ==========")

missing_values = df.isnull().sum()

print(missing_values)

print("\nTotal missing values:",
      missing_values.sum())

# ------------------------------------------------------------
# STEP 2: Handle missing values
# ------------------------------------------------------------

# Separate numerical and categorical columns
numerical_columns = df.select_dtypes(include=np.number).columns.tolist()
categorical_columns = df.select_dtypes(
    include=['object', 'category']
).columns.tolist()

# Remove target variable from feature lists
if 'fraud' in numerical_columns:
    numerical_columns.remove('fraud')

if 'fraud' in categorical_columns:
```

```
    categorical_columns.remove('fraud')

# Fill numerical missing values with median
for column in numerical_columns:
    if df[column].isnull().sum() > 0:
        df[column] = df[column].fillna(df[column].median())

# Fill categorical missing values with mode
for column in categorical_columns:
    if df[column].isnull().sum() > 0:
        df[column] = df[column].fillna(df[column].mode()[0])

print("\n========== MISSING VALUES AFTER HANDLING ==========")
print(df.isnull().sum())

# ------------------------------------------------------------
# STEP 3: Check duplicate records
# ------------------------------------------------------------

print("\n========== DUPLICATE RECORDS ==========")

duplicate_count = df.duplicated().sum()

print("Number of duplicate records:", duplicate_count)

# Remove duplicate records
if duplicate_count > 0:
    df = df.drop_duplicates()
    print("Duplicate records removed.")
else:
    print("No duplicate records found.")

print("Dataset shape after removing duplicates:", df.shape)

# ------------------------------------------------------------
# STEP 4: Identify categorical variables
# ------------------------------------------------------------

print("\n========== CATEGORICAL VARIABLES ==========")

categorical_columns = df.select_dtypes(
    include=['object', 'category']
).columns.tolist()

print(categorical_columns)

# ------------------------------------------------------------
# STEP 5: Identify numerical variables
# ------------------------------------------------------------

print("\n========== NUMERICAL VARIABLES ==========")

numerical_columns = df.select_dtypes(
    include=np.number
```

```
).columns.tolist()

# Remove target variable
if 'fraud' in numerical_columns:
    numerical_columns.remove('fraud')

print(numerical_columns)

# ------------------------------------------------------------
# STEP 6: Separate features and target
# ------------------------------------------------------------

X = df.drop(columns=['fraud'])
y = df['fraud']

print("\n========== FEATURES AND TARGET ==========")

print("Feature shape:", X.shape)
print("Target shape:", y.shape)

print("\nTarget variable:")
print("fraud")

# ------------------------------------------------------------
# STEP 7: One-Hot Encoding
# ------------------------------------------------------------

if len(categorical_columns) > 0:

    print("\n========== APPLYING ONE-HOT ENCODING ==========")

    X = pd.get_dummies(
        X,
        columns=categorical_columns,
        drop_first=True
    )

    print("Categorical variables encoded successfully.")

else:
    print("\nNo categorical variables found.")

# ------------------------------------------------------------
# STEP 8: Convert Boolean columns to integers
# ------------------------------------------------------------

X = X.astype(float)

# ------------------------------------------------------------
# STEP 9: Display final feature variables
# ------------------------------------------------------------

print("\n========== FINAL FEATURE VARIABLES ==========")
```

```
print(X.columns.tolist())

print("\nTotal number of features:", X.shape[1])

# ------------------------------------------------------------
# STEP 10: Verify all features are numerical
# ------------------------------------------------------------

print("\n========== FINAL DATA TYPES ==========")

print(X.dtypes)

# ------------------------------------------------------------
# STEP 11: Verify missing values
# ------------------------------------------------------------

print("\n========== FINAL MISSING VALUE CHECK ==========")

print("Missing values in X:", X.isnull().sum().sum())
print("Missing values in y:", y.isnull().sum())

# ------------------------------------------------------------
# STEP 12: Display final dataset
# ------------------------------------------------------------

print("\n========== FINAL FEATURE DATA ==========")

print(X.head())

print("\n========== TARGET DATA ==========")

print(y.head())

# ------------------------------------------------------------
# STEP 13: Final dataset information
# ------------------------------------------------------------

print("\n========== FINAL DATASET INFORMATION ==========")

print("Features (X):", X.shape)
print("Target (y):", y.shape)

print("\nData preprocessing completed successfully.")
```

```
========== MISSING VALUES ==========
distance_from_home                0
distance_from_last_transaction    0
ratio_to_median_purchase_price    0
repeat_retailer                   0
used_chip                         0
used_pin_number                   0
online_order                      0
fraud                             0
dtype: int64

Total missing values: 0

========== MISSING VALUES AFTER HANDLING ==========
distance_from_home                0
distance_from_last_transaction    0
ratio_to_median_purchase_price    0
repeat_retailer                   0
used_chip                         0
used_pin_number                   0
online_order                      0
fraud                             0
dtype: int64

========== DUPLICATE RECORDS ==========
Number of duplicate records: 0
No duplicate records found.
Dataset shape after removing duplicates: (1000000, 8)

========== CATEGORICAL VARIABLES ==========
[]

========== NUMERICAL VARIABLES ==========
['distance_from_home', 'distance_from_last_transaction', 'ratio_to_media
n_purchase_price', 'repeat_retailer', 'used_chip', 'used_pin_number', 'on
line_order']

========== FEATURES AND TARGET ==========
Feature shape: (1000000, 7)
Target shape: (1000000,)

Target variable:
fraud

No categorical variables found.

========== FINAL FEATURE VARIABLES ==========
['distance_from_home', 'distance_from_last_transaction', 'ratio_to_media
n_purchase_price', 'repeat_retailer', 'used_chip', 'used_pin_number', 'on
line_order']

Total number of features: 7

========== FINAL DATA TYPES ==========
distance_from_home                float64
```

```
distance_from_last_transaction    float64
ratio_to_median_purchase_price    float64
repeat_retailer                   float64
used_chip                         float64
used_pin_number                   float64
online_order                      float64
dtype: object

========== FINAL MISSING VALUE CHECK ==========
Missing values in X: 0
Missing values in y: 0

========== FINAL FEATURE DATA ==========
   distance_from_home  distance_from_last_transaction  \
0           57.877857                        0.311140
1           10.829943                        0.175592
2            5.091079                        0.805153
3            2.247564                        5.600044
4           44.190936                        0.566486

   ratio_to_median_purchase_price  repeat_retailer  used_chip  \
0                        1.945940              1.0        1.0
1                        1.294219              1.0        0.0
2                        0.427715              1.0        0.0
3                        0.362663              1.0        1.0
4                        2.222767              1.0        1.0

   used_pin_number  online_order
0              0.0           0.0
1              0.0           0.0
2              0.0           1.0
3              0.0           1.0
4              0.0           1.0

========== TARGET DATA ==========
0    0.0
1    0.0
2    0.0
3    0.0
4    0.0
Name: fraud, dtype: float64

========== FINAL DATASET INFORMATION ==========
Features (X): (1000000, 7)
Target (y): (1000000,)

Data preprocessing completed successfully.
```

# Task 4: Feature Scaling and Class Imbalance Handling

Standardize the numerical features using **Standardization** so that they have

approximately zero mean and unit variance.

The standardized value of a feature is calculated as:

$$ z = \frac{x-\mu}{\sigma} $$
where:

- (x) = original feature value
- (\mu) = mean of the feature
- (\sigma) = standard deviation of the feature
- (z) = standardized feature value

Handle the **class imbalance** in the dataset using a suitable technique such as:

- Random Undersampling
- Oversampling
- Class Weighting

Display the class distribution **before and after** applying the selected class-imbalance handling technique.

Compare the number of legitimate and fraudulent transactions before and after balancing.

The objective of this task is to ensure that the features are on a comparable scale and that the Logistic Regression model does not become biased toward the majority class.

In [5]:
```
# ========================================
# TASK 4: FEATURE SCALING AND CLASS IMBALANCE
# ========================================

from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

# ----------------------------------------
# 1. Separate Features and Target
# ----------------------------------------

X = df.drop(columns=['fraud'])
y = df['fraud']

print("Original Dataset Shape:")
print("X:", X.shape)
print("y:", y.shape)
```

```python
# ----------------------------------------
# 2. Display Class Distribution BEFORE
# ----------------------------------------

print("\n===== CLASS DISTRIBUTION BEFORE BALANCING =====")

before_distribution = y.value_counts().sort_index()

print("Legitimate Transactions (0):", before_distribution.get(0, 0))
print("Fraudulent Transactions (1):", before_distribution.get(1, 0))


# Calculate percentages
before_percentage = y.value_counts(normalize=True).sort_index() * 100

print("\nPercentage:")
print("Legitimate (0): {:.2f}%".format(before_percentage.get(0, 0)))
print("Fraudulent (1): {:.2f}%".format(before_percentage.get(1, 0)))


# ----------------------------------------
# 3. Split Dataset into Training and Testing
# ----------------------------------------

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("\n===== DATA SPLIT =====")
print("Training Data:", X_train.shape)
print("Testing Data:", X_test.shape)


# ----------------------------------------
# 4. Feature Scaling
# ----------------------------------------

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("\n===== FEATURE SCALING =====")
print("Standardization completed successfully.")

print("\nMean of first 5 standardized features:")
print(X_train_scaled[:, :5].mean(axis=0))
```

```
print("\nStandard deviation of first 5 standardized features:")
print(X_train_scaled[:, :5].std(axis=0))


# ----------------------------------------
# 5. Handle Class Imbalance
# Using Class Weighting
# ----------------------------------------

model = LogisticRegression(
    class_weight='balanced',
    max_iter=1000,
    random_state=42
)

print("\n===== CLASS IMBALANCE HANDLING =====")
print("Technique Used: Class Weighting")
print("class_weight = 'balanced'")


# ----------------------------------------
# 6. Display Class Distribution AFTER
# ----------------------------------------

# Class weighting does not change the number of records.
# It changes the importance/weight given to each class during training.

after_distribution = y_train.value_counts().sort_index()

print("\n===== CLASS DISTRIBUTION AFTER BALANCING =====")

print("Legitimate Transactions (0):",
      after_distribution.get(0, 0))

print("Fraudulent Transactions (1):",
      after_distribution.get(1, 0))

print("\nNote: Class weighting does not physically change the number")
print("of legitimate or fraudulent transactions.")


# ----------------------------------------
# 7. Calculate Class Weights
# ----------------------------------------

from sklearn.utils.class_weight import compute_class_weight

classes = np.unique(y_train)

weights = compute_class_weight(
    class_weight='balanced',
    classes=classes,
    y=y_train
)
```

```
class_weights = dict(zip(classes, weights))

print("\n===== CALCULATED CLASS WEIGHTS =====")
print(class_weights)
```

```
Original Dataset Shape:
X: (1000000, 7)
y: (1000000,)

===== CLASS DISTRIBUTION BEFORE BALANCING =====
Legitimate Transactions (0): 912597
Fraudulent Transactions (1): 87403

Percentage:
Legitimate (0): 91.26%
Fraudulent (1): 8.74%

===== DATA SPLIT =====
Training Data: (800000, 7)
Testing Data: (200000, 7)

===== FEATURE SCALING =====
Standardization completed successfully.

Mean of first 5 standardized features:
[ 9.56124069e-17  3.49231755e-17 -8.73612294e-17  1.79873894e-16
  6.09290396e-17]

Standard deviation of first 5 standardized features:
[1. 1. 1. 1. 1.]

===== CLASS IMBALANCE HANDLING =====
Technique Used: Class Weighting
class_weight = 'balanced'

===== CLASS DISTRIBUTION AFTER BALANCING =====
Legitimate Transactions (0): 730078
Fraudulent Transactions (1): 69922

Note: Class weighting does not physically change the number
of legitimate or fraudulent transactions.

===== CALCULATED CLASS WEIGHTS =====
{np.float64(0.0): np.float64(0.5478866641646509), np.float64(1.0): np.flo
at64(5.720660164182947)}
```

# Task 5: Split the Dataset

Divide the preprocessed dataset into three subsets:

- **Training Set -- 60%**

- **Validation Set -- 20%**
- **Test Set -- 20%**

Use **stratified splitting** to maintain a representative proportion of fraudulent and legitimate transactions in each subset.

The dataset should first be divided into training and temporary sets. The temporary set should then be divided equally into validation and test sets.

The objective of this task is to use the training set for model training, the validation set for model evaluation and parameter selection, and the test set for final performance evaluation.

In [6]:
```
# ========================================
# TASK 5: SPLIT THE DATASET
# ========================================

from sklearn.model_selection import train_test_split

# Features and target
X = df.drop(columns=['fraud'])
y = df['fraud']

# ----------------------------------------
# Step 1: Split into Training (60%)
# and Temporary (40%)
# ----------------------------------------

X_train, X_temp, y_train, y_temp = train_test_split(
    X,
    y,
    test_size=0.40,
    random_state=42,
    stratify=y
)

# ----------------------------------------
# Step 2: Split Temporary Set into
# Validation (20%) and Test (20%)
# ----------------------------------------

X_val, X_test, y_val, y_test = train_test_split(
    X_temp,
    y_temp,
    test_size=0.50,
    random_state=42,
    stratify=y_temp
)

# ----------------------------------------
# Display Dataset Sizes
```

```python
# ----------------------------------------

print("===== DATASET SPLIT =====")

print("\nTraining Set:")
print("Features:", X_train.shape)
print("Target:", y_train.shape)

print("\nValidation Set:")
print("Features:", X_val.shape)
print("Target:", y_val.shape)

print("\nTest Set:")
print("Features:", X_test.shape)
print("Target:", y_test.shape)

# ----------------------------------------
# Display Percentages
# ----------------------------------------

total = len(df)

print("\n===== PERCENTAGE OF DATA =====")

print("Training Set: {:.2f}%".format(len(X_train) / total * 100))
print("Validation Set: {:.2f}%".format(len(X_val) / total * 100))
print("Test Set: {:.2f}%".format(len(X_test) / total * 100))

# ----------------------------------------
# Check Stratification
# ----------------------------------------

print("\n===== CLASS DISTRIBUTION =====")

print("\nTraining Set:")
print(y_train.value_counts(normalize=True) * 100)

print("\nValidation Set:")
print(y_val.value_counts(normalize=True) * 100)

print("\nTest Set:")
print(y_test.value_counts(normalize=True) * 100)
```

```
===== DATASET SPLIT =====

Training Set:
Features: (600000, 7)
Target: (600000,)

Validation Set:
Features: (200000, 7)
Target: (200000,)

Test Set:
Features: (200000, 7)
Target: (200000,)

===== PERCENTAGE OF DATA =====
Training Set: 60.00%
Validation Set: 20.00%
Test Set: 20.00%

===== CLASS DISTRIBUTION =====

Training Set:
fraud
0.0    91.259667
1.0     8.740333
Name: proportion, dtype: float64

Validation Set:
fraud
0.0    91.2595
1.0     8.7405
Name: proportion, dtype: float64

Test Set:
fraud
0.0    91.26
1.0     8.74
Name: proportion, dtype: float64
```

# Task 6: Implement Sigmoid and Cost Functions

Implement the **Sigmoid Function** used in Logistic Regression.

The Sigmoid Function is defined as:

$$ \sigma(z) = \frac{1}{1+e^{-z}} $$

The Sigmoid Function converts the output of the linear equation into a probability value between 0 and 1.

Implement the **Binary Cross-Entropy Cost Function**:

$$ J(w,b) = -\frac{1}{m} \sum_{i=1}^{m} \left[ y_i\log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i) \right] $$
where:

- (m) = number of training samples
- (y_i) = actual class label
- (\hat{y}_i) = predicted probability
- (w) = weight vector
- (b) = bias

Test the Sigmoid Function using suitable sample values.

Calculate the initial cost of the Logistic Regression model using the initialized weights and bias.

The objective of this task is to understand how the Sigmoid Function produces probabilities and how the Binary Cross-Entropy Cost Function measures the error of the Logistic Regression model.

In [7]:
```
# TASK 6: IMPLEMENT SIGMOID AND COST FUNCTIONS
# Logistic Regression - Credit Card Fraud Detection
# ============================================================

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# ------------------------------------------------------------
# STEP 1: Convert training data to NumPy arrays
# ------------------------------------------------------------

X_train_np = np.asarray(X_train, dtype=float)
y_train_np = np.asarray(y_train, dtype=float)

print("Training feature shape:", X_train_np.shape)
print("Training target shape:", y_train_np.shape)


# ------------------------------------------------------------
# STEP 2: Define Sigmoid Function
# ------------------------------------------------------------

def sigmoid(z):
    """
    Sigmoid function:

            1
```

```
    σ(z) = ------
           1 + e^(-z)

    Converts any real number into a value between 0 and 1.
    """

    # Prevent overflow in exponential calculation
    z = np.clip(z, -500, 500)

    return 1 / (1 + np.exp(-z))


# ------------------------------------------------------------
# STEP 3: Test Sigmoid Function
# ------------------------------------------------------------

print("\n========== SIGMOID FUNCTION TEST ==========")

sample_values = np.array([-10, -5, -2, 0, 2, 5, 10])

sigmoid_values = sigmoid(sample_values)

for z, result in zip(sample_values, sigmoid_values):
    print("z =", z, "  sigmoid(z) =", round(result, 6))


# ------------------------------------------------------------
# STEP 4: Plot Sigmoid Function
# ------------------------------------------------------------

z_values = np.linspace(-10, 10, 100)

plt.figure(figsize=(8, 5))
plt.plot(z_values, sigmoid(z_values))

plt.xlabel("z")
plt.ylabel("Sigmoid(z)")
plt.title("Sigmoid Function")

plt.grid(True)
plt.show()


# ------------------------------------------------------------
# STEP 5: Initialize weights and bias
# ------------------------------------------------------------

number_of_features = X_train_np.shape[1]

# Initialize all weights to zero
w = np.zeros(number_of_features)

# Initialize bias to zero
b = 0.0
```

```
print("\n========== INITIAL PARAMETERS ==========")

print("Number of features:", number_of_features)
print("Initial weights:")
print(w)

print("Initial bias:", b)


# ------------------------------------------------------------
# STEP 6: Calculate predicted probabilities
# ------------------------------------------------------------

def predict_probability(X, w, b):
    """
    Calculate predicted probability:

    z = Xw + b
    probability = sigmoid(z)
    """

    z = np.dot(X, w) + b

    return sigmoid(z)


y_pred_initial = predict_probability(
    X_train_np,
    w,
    b
)

print("\n========== INITIAL PREDICTIONS ==========")

print("First 10 predicted probabilities:")
print(y_pred_initial[:10])


# ------------------------------------------------------------
# STEP 7: Implement Binary Cross-Entropy Cost Function
# ------------------------------------------------------------

def compute_cost(X, y, w, b):
    """
    Binary Cross-Entropy Cost:

    J(w,b) = -1/m Σ[
             y log(y_hat)
             +
             (1-y) log(1-y_hat)
             ]
    """
```

```
    m = len(y)

    # Calculate predicted probabilities
    y_hat = predict_probability(X, w, b)

    # Prevent log(0)
    epsilon = 1e-15

    y_hat = np.clip(
        y_hat,
        epsilon,
        1 - epsilon
    )

    # Binary Cross-Entropy
    cost = -(1 / m) * np.sum(
        y * np.log(y_hat)
        +
        (1 - y) * np.log(1 - y_hat)
    )

    return cost


# ------------------------------------------------------------
# STEP 8: Calculate Initial Cost
# ------------------------------------------------------------

initial_cost = compute_cost(
    X_train_np,
    y_train_np,
    w,
    b
)

print("\n========== INITIAL COST ==========")

print("Initial Cost:", initial_cost)

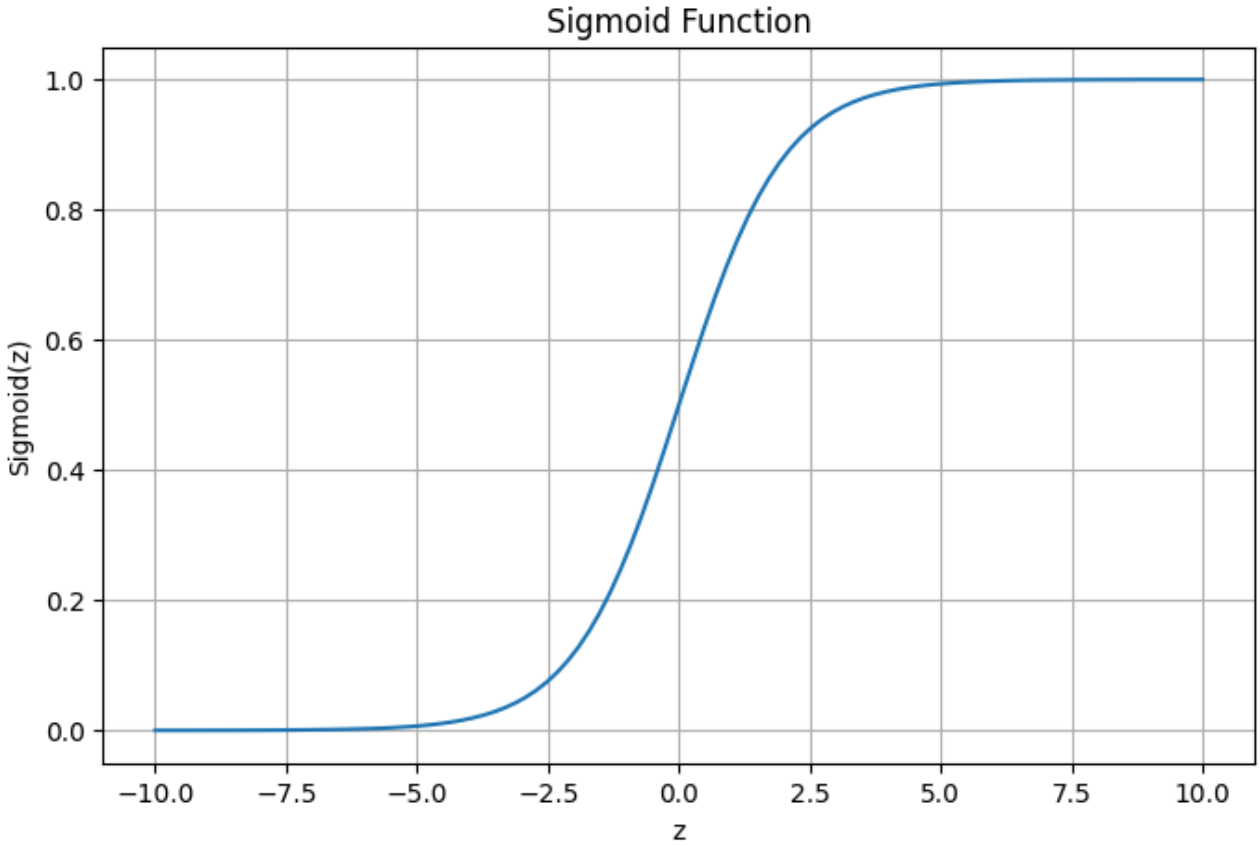

# ------------------------------------------------------------
# STEP 9: Display complete result
# ------------------------------------------------------------

print("\n========== TASK 6 SUMMARY ==========")

print("Sigmoid function implemented successfully.")
print("Binary Cross-Entropy cost function implemented successfully.")
print("Weights initialized to zero.")
print("Bias initialized to zero.")
print("Initial cost calculated successfully.")
```

```
Training feature shape: (600000, 7)
Training target shape: (600000,)

========== SIGMOID FUNCTION TEST ==========
z = -10   sigmoid(z) = 4.5e-05
z = -5   sigmoid(z) = 0.006693
z = -2   sigmoid(z) = 0.119203
z = 0   sigmoid(z) = 0.5
z = 2   sigmoid(z) = 0.880797
z = 5   sigmoid(z) = 0.993307
z = 10   sigmoid(z) = 0.999955
```



```
========== INITIAL PARAMETERS ==========
Number of features: 7
Initial weights:
[0. 0. 0. 0. 0. 0. 0.]
Initial bias: 0.0

========== INITIAL PREDICTIONS ==========
First 10 predicted probabilities:
[0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5]

========== INITIAL COST ==========
Initial Cost: 0.6931471805599454

========== TASK 6 SUMMARY ==========
Sigmoid function implemented successfully.
Binary Cross-Entropy cost function implemented successfully.
Weights initialized to zero.
Bias initialized to zero.
Initial cost calculated successfully.
```

# Task 7: Implement Logistic Regression Using Gradient Descent

Implement **Logistic Regression from scratch** using NumPy without directly using the Scikit-learn Logistic Regression implementation.

Initialize the model parameters, including the **weight vector** (w) and **bias** (b).

Calculate the predicted probability using the Sigmoid Function:

$$ \hat{y} = \sigma(Xw+b) $$
where:

- (X) = input feature matrix
- (w) = weight vector
- $\hat{y} = \text{predicted probability}$

Calculate the gradients of the cost function with respect to the weights and bias:

$$ \frac{\partial J}{\partial w} = \frac{1}{m}X^T(\hat{y}-y) $$ $$ \frac{\partial J}{\partial b} = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}_i-y_i) $$
Update the model parameters using **Gradient Descent**:

$$ w := w-\alpha\frac{\partial J}{\partial w} $$$$ b := b-\alpha\frac{\partial J}{\partial b} $$

where ($\alpha$) represents the **learning rate**.

Train the model for a suitable number of iterations and record the cost value during each iteration.

Plot **Cost vs. Iteration** to verify whether the optimization process is converging.

The objective of this task is to implement Logistic Regression manually and understand how Gradient Descent optimizes the model parameters by minimizing the Binary Cross-Entropy Cost Function.

In [8]:
```
# ============================================================
# TASK 7: IMPLEMENT LOGISTIC REGRESSION USING GRADIENT DESCENT
# ============================================================

import numpy as np
import matplotlib.pyplot as plt

# ------------------------------------------------------------
# 1. Scale Training, Validation and Test Data
# ------------------------------------------------------------

# Fit scaler only on training data
X_train_scaled = scaler.fit_transform(X_train)

# Use the same scaler for validation and test data
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)

# Convert to NumPy arrays
X_train_np = np.asarray(X_train_scaled, dtype=float)
X_val_np = np.asarray(X_val_scaled, dtype=float)
X_test_np = np.asarray(X_test_scaled, dtype=float)

y_train_np = np.asarray(y_train, dtype=float)
y_val_np = np.asarray(y_val, dtype=float)
y_test_np = np.asarray(y_test, dtype=float)


# ------------------------------------------------------------
# 2. Sigmoid Function
# ------------------------------------------------------------

def sigmoid(z):
    return 1 / (1 + np.exp(-z))


# ------------------------------------------------------------
# 3. Binary Cross-Entropy Cost Function
# ------------------------------------------------------------
```

```python
def compute_cost(X, y, w, b):

    m = len(y)

    # Linear equation
    z = np.dot(X, w) + b

    # Predicted probability
    y_pred = sigmoid(z)

    # Avoid log(0)
    epsilon = 1e-15
    y_pred = np.clip(y_pred, epsilon, 1 - epsilon)

    # Binary Cross-Entropy Cost
    cost = -(1 / m) * np.sum(
        y * np.log(y_pred) +
        (1 - y) * np.log(1 - y_pred)
    )

    return cost


# ------------------------------------------------------------
# 4. Gradient Descent Function
# ------------------------------------------------------------

def gradient_descent(X, y, learning_rate=0.01, iterations=1000):

    m, n = X.shape

    # Initialize weights and bias
    w = np.zeros(n)
    b = 0.0

    # Store cost at every iteration
    cost_history = []

    # Gradient Descent
    for i in range(iterations):

        # Calculate z
        z = np.dot(X, w) + b

        # Calculate predicted probability
        y_pred = sigmoid(z)

        # Calculate gradients
        dw = (1 / m) * np.dot(X.T, (y_pred - y))
        db = (1 / m) * np.sum(y_pred - y)

        # Update weights
        w = w - learning_rate * dw
```

```
        # Update bias
        b = b - learning_rate * db

        # Calculate cost
        cost = compute_cost(X, y, w, b)

        # Store cost
        cost_history.append(cost)

    return w, b, cost_history


# ------------------------------------------------------------
# 5. Set Learning Rate and Iterations
# ------------------------------------------------------------

learning_rate = 0.01
iterations = 1000


# ------------------------------------------------------------
# 6. Train Logistic Regression Model
# ------------------------------------------------------------

w, b, cost_history = gradient_descent(
    X_train_np,
    y_train_np,
    learning_rate,
    iterations
)


# ------------------------------------------------------------
# 7. Display Model Parameters
# ------------------------------------------------------------

print("================================================")
print("LOGISTIC REGRESSION TRAINING")
print("================================================")

print("\nLearning Rate:", learning_rate)
print("Number of Iterations:", iterations)

print("\nNumber of Features:", len(w))

print("\nFinal Bias:")
print(b)

print("\nFirst 10 Weights:")
print(w[:10])

print("\nInitial Cost:")
print(cost_history[0])
```

```python
print("\nFinal Cost:")
print(cost_history[-1])


# ------------------------------------------------------------
# 8. Plot Cost vs Iteration
# ------------------------------------------------------------

plt.figure(figsize=(8, 5))

plt.plot(
    range(1, iterations + 1),
    cost_history
)

plt.xlabel("Iteration")
plt.ylabel("Cost")
plt.title("Cost vs Iteration")

plt.grid(True)
plt.show()


# ------------------------------------------------------------
# 9. Prediction Probability Function
# ------------------------------------------------------------

def predict_probability(X, w, b):

    z = np.dot(X, w) + b

    probability = sigmoid(z)

    return probability


# ------------------------------------------------------------
# 10. Prediction Class Function
# ------------------------------------------------------------

def predict_class(X, w, b, threshold=0.5):

    probability = predict_probability(X, w, b)

    prediction = (probability >= threshold).astype(int)

    return prediction


# ------------------------------------------------------------
# 11. Training Predictions
# ------------------------------------------------------------
```

```
train_probability = predict_probability(
    X_train_np,
    w,
    b
)

train_prediction = predict_class(
    X_train_np,
    w,
    b
)


# ------------------------------------------------------------
# 12. Validation Predictions
# ------------------------------------------------------------

val_probability = predict_probability(
    X_val_np,
    w,
    b
)

val_prediction = predict_class(
    X_val_np,
    w,
    b
)


# ------------------------------------------------------------
# 13. Test Predictions
# ------------------------------------------------------------

test_probability = predict_probability(
    X_test_np,
    w,
    b
)

test_prediction = predict_class(
    X_test_np,
    w,
    b
)


# ------------------------------------------------------------
# 14. Display Sample Predictions
# ------------------------------------------------------------

print("\n================================================")
print("SAMPLE PREDICTIONS")
print("================================================")
```

```python
print("\nFirst 10 Predicted Probabilities:")
print(test_probability[:10])

print("\nFirst 10 Predicted Classes:")
print(test_prediction[:10])

print("\nFirst 10 Actual Classes:")
print(y_test_np[:10])


# ------------------------------------------------------------
# 15. Display Prediction Distribution
# ------------------------------------------------------------

print("\n================================================")
print("TEST PREDICTION DISTRIBUTION")
print("================================================")

print(
    "Legitimate Predictions (0):",
    np.sum(test_prediction == 0)
)

print(
    "Fraud Predictions (1):",
    np.sum(test_prediction == 1)
)
```

```
================================================
LOGISTIC REGRESSION TRAINING
================================================

Learning Rate: 0.01
Number of Iterations: 1000

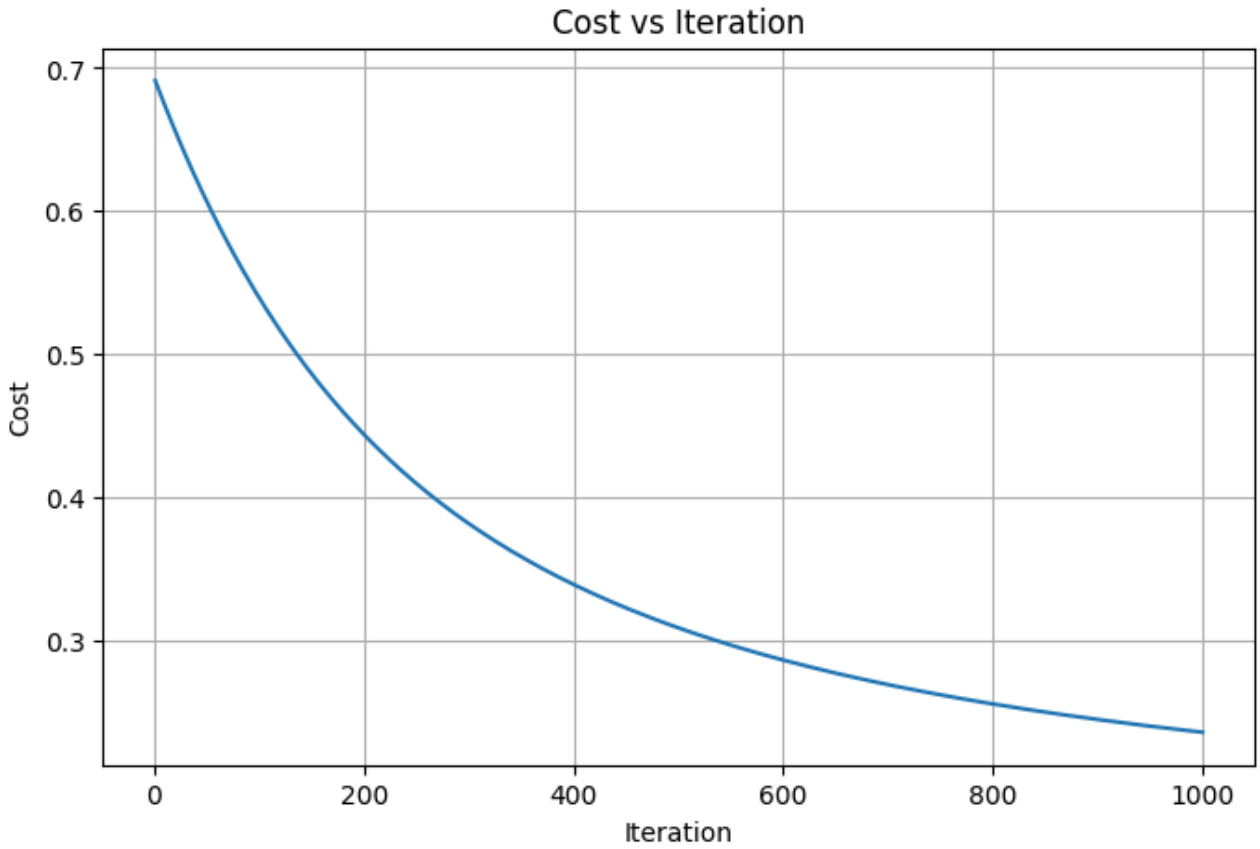
Number of Features: 7

Final Bias:
-1.753268106189117

First 10 Weights:
[ 0.26463744  0.14350385  0.63899969 -0.02414737 -0.09132464 -0.15469399
  0.28821672]

Initial Cost:
0.6912070193730956

Final Cost:
0.2359747593437483
```




```
================================================
SAMPLE PREDICTIONS
================================================

First 10 Predicted Probabilities:
[0.08123795 0.14056    0.34049511 0.12004904 0.17783067 0.15342214
 0.12947607 0.09476524 0.23856133 0.15449594]

First 10 Predicted Classes:
[0 0 0 0 0 0 0 0 0 0]

First 10 Actual Classes:
[0. 0. 1. 0. 0. 0. 0. 0. 0. 0.]

================================================
TEST PREDICTION DISTRIBUTION
================================================
Legitimate Predictions (0): 195148
Fraud Predictions (1): 4852
```

# Task 8: Generate Predictions and Evaluate the Confusion Matrix

Use the trained Logistic Regression model to calculate the **fraud**

**probabilities** for the validation and test datasets.

The predicted probability is calculated using:

$$ \hat{y} = \sigma(Xw+b) $$
Convert the predicted probabilities into class labels using a threshold of **0.5**:

$$ \hat{y}_{class} = \begin{cases} 1, & \text{if } \hat{y} \geq 0.5 \ 0, & \text{if } \hat{y} < 0.5 \end{cases} $$
Construct the **Confusion Matrix** and identify the following:

- **True Positive (TP):** Fraudulent transactions correctly classified as fraudulent.
- **True Negative (TN):** Legitimate transactions correctly classified as legitimate.
- **False Positive (FP):** Legitimate transactions incorrectly classified as fraudulent.
- **False Negative (FN):** Fraudulent transactions incorrectly classified as legitimate.

The confusion matrix can be represented as:

$$ \text{Confusion Matrix} = \begin{bmatrix} TN & FP \\ FN & TP \end{bmatrix} $$
Explain the meaning of each result in the context of **credit card fraud detection**.

The objective of this task is to evaluate the classification results of the Logistic Regression model and understand the different types of prediction errors, with particular attention to **False Negatives**, which represent fraudulent transactions that are not detected.

In [9]:
```
# ============================================================
# TASK 8: GENERATE PREDICTIONS AND CONFUSION MATRIX
# ============================================================

import numpy as np
import matplotlib.pyplot as plt


# ------------------------------------------------------------
# 1. Calculate Fraud Probabilities
# ------------------------------------------------------------

def predict_probability(X, w, b):
    z = np.dot(X, w) + b
```

```
    return sigmoid(z)


# Validation probabilities
val_probability = predict_probability(
    X_val_np,
    w,
    b
)

# Test probabilities
test_probability = predict_probability(
    X_test_np,
    w,
    b
)


# ------------------------------------------------------------
# 2. Convert Probabilities into Class Labels
# ------------------------------------------------------------

def predict_class(X, w, b, threshold=0.5):

    probability = predict_probability(X, w, b)

    return (probability >= threshold).astype(int)


# Validation predictions
val_prediction = predict_class(
    X_val_np,
    w,
    b,
    threshold=0.5
)

# Test predictions
test_prediction = predict_class(
    X_test_np,
    w,
    b,
    threshold=0.5
)


# ------------------------------------------------------------
# 3. Display Sample Probabilities and Predictions
# ------------------------------------------------------------

print("================================================")
print("VALIDATION DATA")
print("================================================")
```

```
print("\nFirst 10 Fraud Probabilities:")
print(val_probability[:10])

print("\nFirst 10 Predicted Classes:")
print(val_prediction[:10])

print("\nFirst 10 Actual Classes:")
print(y_val_np[:10])


print("\n==================================================")
print("TEST DATA")
print("==================================================")

print("\nFirst 10 Fraud Probabilities:")
print(test_probability[:10])

print("\nFirst 10 Predicted Classes:")
print(test_prediction[:10])

print("\nFirst 10 Actual Classes:")
print(y_test_np[:10])


# ------------------------------------------------------------
# 4. Calculate Confusion Matrix Manually
# ------------------------------------------------------------

def calculate_confusion_matrix(y_actual, y_predicted):

    TN = np.sum((y_actual == 0) & (y_predicted == 0))
    FP = np.sum((y_actual == 0) & (y_predicted == 1))
    FN = np.sum((y_actual == 1) & (y_predicted == 0))
    TP = np.sum((y_actual == 1) & (y_predicted == 1))

    return TN, FP, FN, TP


# Validation confusion matrix
val_TN, val_FP, val_FN, val_TP = calculate_confusion_matrix(
    y_val_np,
    val_prediction
)

# Test confusion matrix
test_TN, test_FP, test_FN, test_TP = calculate_confusion_matrix(
    y_test_np,
    test_prediction
)


# ------------------------------------------------------------
# 5. Display Validation Confusion Matrix
# ------------------------------------------------------------
```

```
print("\n================================================")
print("VALIDATION CONFUSION MATRIX")
print("================================================")

print("\n              Predicted")
print("              Legitimate   Fraud")
print(
    "Actual Legitimate    ",
    val_TN,
    "       ",
    val_FP
)

print(
    "Actual Fraudulent    ",
    val_FN,
    "       ",
    val_TP
)

print("\nTN =", val_TN)
print("FP =", val_FP)
print("FN =", val_FN)
print("TP =", val_TP)


# ------------------------------------------------------------
# 6. Display Test Confusion Matrix
# ------------------------------------------------------------

print("\n================================================")
print("TEST CONFUSION MATRIX")
print("================================================")

print("\n              Predicted")
print("              Legitimate   Fraud")

print(
    "Actual Legitimate    ",
    test_TN,
    "       ",
    test_FP
)

print(
    "Actual Fraudulent    ",
    test_FN,
    "       ",
    test_TP
)

print("\nTN =", test_TN)
print("FP =", test_FP)
```

```
print("FN =", test_FN)
print("TP =", test_TP)


# ------------------------------------------------------------
# 7. Create Confusion Matrix Array
# ------------------------------------------------------------

test_confusion_matrix = np.array([
    [test_TN, test_FN],
    [test_FP, test_TP]
])

print("\nConfusion Matrix:")
print(test_confusion_matrix)


# ------------------------------------------------------------
# 8. Visualize Confusion Matrix
# ------------------------------------------------------------

plt.figure(figsize=(6, 5))

plt.imshow(test_confusion_matrix)

plt.title("Confusion Matrix - Test Dataset")
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")

plt.xticks(
    [0, 1],
    ["Legitimate (0)", "Fraudulent (1)"]
)

plt.yticks(
    [0, 1],
    ["Legitimate (0)", "Fraudulent (1)"]
)

# Display values inside matrix
for i in range(2):
    for j in range(2):
        plt.text(
            j,
            i,
            test_confusion_matrix[i, j],
            ha="center",
            va="center"
        )

plt.colorbar()
plt.show()
```

```
================================================
VALIDATION DATA
================================================

First 10 Fraud Probabilities:
[0.15915915 0.11744334 0.08241025 0.12812786 0.13558322 0.14255971
 0.09504991 0.22724923 0.07640216 0.15244208]

First 10 Predicted Classes:
[0 0 0 0 0 0 0 0 0 0]

First 10 Actual Classes:
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]

================================================
TEST DATA
================================================

First 10 Fraud Probabilities:
[0.08123795 0.14056    0.34049511 0.12004904 0.17783067 0.15342214
 0.12947607 0.09476524 0.23856133 0.15449594]

First 10 Predicted Classes:
[0 0 0 0 0 0 0 0 0 0]

First 10 Actual Classes:
[0. 0. 1. 0. 0. 0. 0. 0. 0. 0.]

================================================
VALIDATION CONFUSION MATRIX
================================================

              Predicted
              Legitimate   Fraud
Actual Legitimate     181277         1242
Actual Fraudulent     13707         3774

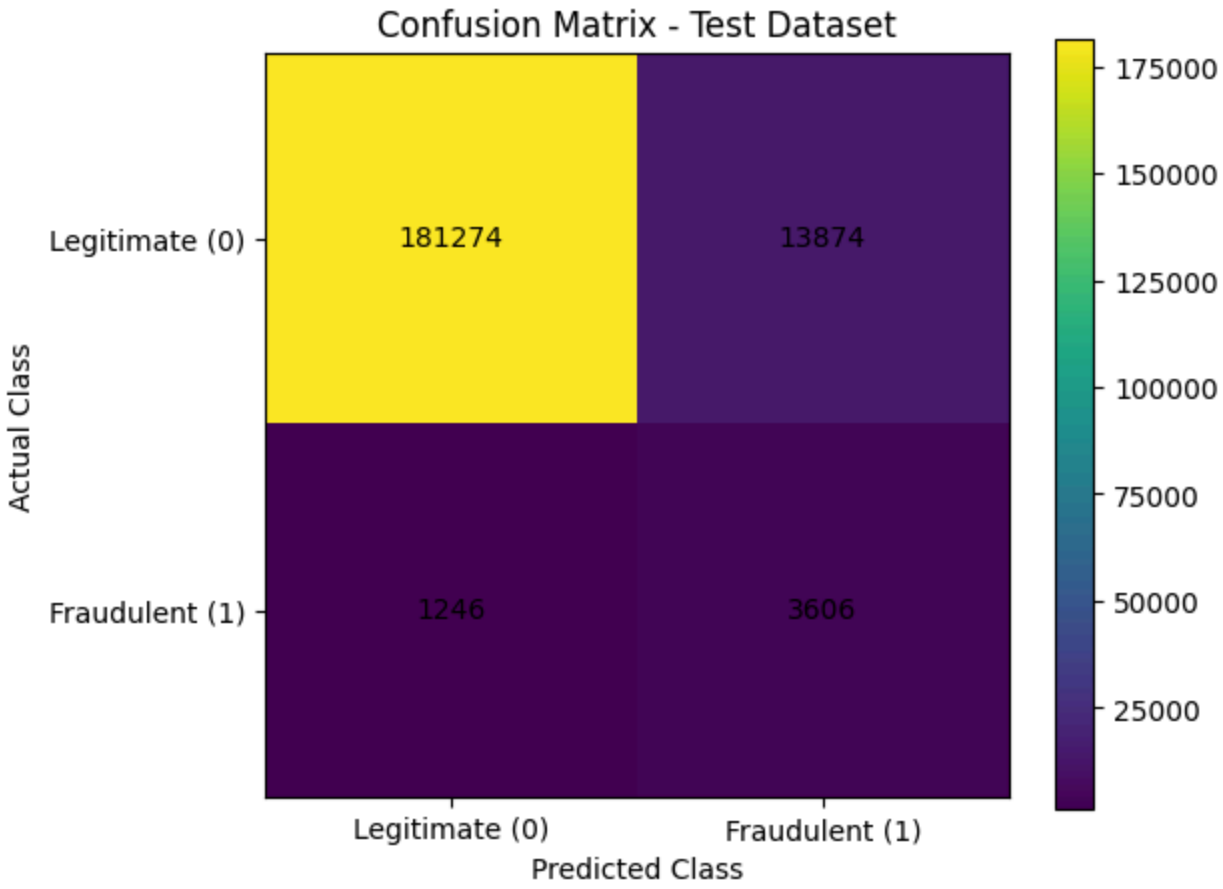
TN = 181277
FP = 1242
FN = 13707
TP = 3774

================================================
TEST CONFUSION MATRIX
================================================

              Predicted
              Legitimate   Fraud
Actual Legitimate     181274         1246
Actual Fraudulent     13874         3606

TN = 181274
FP = 1246
FN = 13874
TP = 3606
```

```
Confusion Matrix:
[[181274  13874]
 [  1246   3606]]
```




# Task 9: Calculate Precision, Recall and F1-Score

Calculate the following performance metrics for the Logistic Regression model:

## Precision

Precision measures the proportion of transactions predicted as fraudulent that are actually fraudulent.

$$ \text{Precision} = \frac{TP}{TP+FP} $$

## Recall

Recall measures the proportion of actual fraudulent transactions that are correctly identified by the model.

$$ \text{Recall} = \frac{TP}{TP+FN} $$

## F1-Score

The F1-Score is the harmonic mean of Precision and Recall.

$$ F1\text{-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision}+\text{Recall}} $$

## Accuracy

Calculate accuracy for comparison:

$$ \text{Accuracy} = \frac{TP+TN} {TP+TN+FP+FN} $$
Display the calculated **Accuracy, Precision, Recall, and F1-Score**.

Explain why **Recall is more important than accuracy** in many fraud detection applications. A False Negative represents a fraudulent transaction that is incorrectly classified as legitimate. Therefore, a model with high accuracy but low Recall may fail to detect a significant number of fraudulent transactions.

The objective of this task is to evaluate the Logistic Regression model using appropriate classification metrics and understand the importance of Recall in credit card fraud detection.

In [10…

```python
# ============================================================
# TASK 9: CALCULATE PRECISION, RECALL, F1-SCORE AND ACCURACY
# ============================================================

import numpy as np
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)

# ------------------------------------------------------------
# 1. Get Predictions
# ------------------------------------------------------------
```

```python
# Make sure these variables are available from Task 7
# w, b, X_test_np, y_test_np

# Calculate fraud probabilities
test_probability = sigmoid(
    np.dot(X_test_np, w) + b
)

# Convert probabilities into class labels
# Threshold = 0.5
test_prediction = (test_probability >= 0.5).astype(int)


# ------------------------------------------------------------
# 2. Calculate Confusion Matrix Values
# ------------------------------------------------------------

TN = np.sum(
    (y_test_np == 0) & (test_prediction == 0)
)

FP = np.sum(
    (y_test_np == 0) & (test_prediction == 1)
)

FN = np.sum(
    (y_test_np == 1) & (test_prediction == 0)
)

TP = np.sum(
    (y_test_np == 1) & (test_prediction == 1)
)


# ------------------------------------------------------------
# 3. Calculate Accuracy
# ------------------------------------------------------------

accuracy = (
    (TP + TN) /
    (TP + TN + FP + FN)
)


# ------------------------------------------------------------
# 4. Calculate Precision
# ------------------------------------------------------------

if (TP + FP) != 0:
    precision = TP / (TP + FP)
else:
    precision = 0
```

```
# ------------------------------------------------------------
# 5. Calculate Recall
# ------------------------------------------------------------

if (TP + FN) != 0:
    recall = TP / (TP + FN)
else:
    recall = 0


# ------------------------------------------------------------
# 6. Calculate F1-Score
# ------------------------------------------------------------

if (precision + recall) != 0:
    f1 = (
        2 * precision * recall
        / (precision + recall)
    )
else:
    f1 = 0


# ------------------------------------------------------------
# 7. Display Confusion Matrix Values
# ------------------------------------------------------------

print("================================================")
print("CONFUSION MATRIX VALUES")
print("================================================")

print("True Negative  (TN):", TN)
print("False Positive (FP):", FP)
print("False Negative (FN):", FN)
print("True Positive  (TP):", TP)


# ------------------------------------------------------------
# 8. Display Performance Metrics
# ------------------------------------------------------------

print("\n================================================")
print("MODEL PERFORMANCE")
print("================================================")

print(
    "Accuracy  : {:.4f} ({:.2f}%)".format(
        accuracy, accuracy * 100
    )
)

print(
    "Precision : {:.4f} ({:.2f}%)".format(
```

```
        precision, precision * 100
    )
)

print(
    "Recall    : {:.4f} ({:.2f}%)".format(
        recall, recall * 100
    )
)

print(
    "F1-Score  : {:.4f} ({:.2f}%)".format(
        f1, f1 * 100
    )
)


# ------------------------------------------------------------
# 9. Verify Results Using Scikit-learn
# ------------------------------------------------------------

sk_accuracy = accuracy_score(
    y_test_np,
    test_prediction
)

sk_precision = precision_score(
    y_test_np,
    test_prediction,
    zero_division=0
)

sk_recall = recall_score(
    y_test_np,
    test_prediction,
    zero_division=0
)

sk_f1 = f1_score(
    y_test_np,
    test_prediction,
    zero_division=0
)


print("\n================================================")
print("SCIKIT-LEARN VERIFICATION")
print("================================================")

print("Accuracy  :", sk_accuracy)
print("Precision :", sk_precision)
print("Recall    :", sk_recall)
print("F1-Score  :", sk_f1)
```

```python
# ------------------------------------------------------------
# 10. Interpretation
# ------------------------------------------------------------

print("\n================================================")
print("INTERPRETATION")
print("================================================")

print("Accuracy  : Overall percentage of correct predictions.")
print("Precision : Percentage of predicted frauds that are actually fraud
print("Recall    : Percentage of actual frauds detected by the model.")
print("F1-Score  : Balance between Precision and Recall.")

print("\nImportant:")
print("Recall is especially important in fraud detection")
print("because False Negatives represent fraudulent")
print("transactions that were incorrectly classified as legitimate.")
```

```
================================================
CONFUSION MATRIX VALUES
================================================
True Negative  (TN): 181274
False Positive (FP): 1246
False Negative (FN): 13874
True Positive  (TP): 3606

================================================
MODEL PERFORMANCE
================================================
Accuracy  : 0.9244 (92.44%)
Precision : 0.7432 (74.32%)
Recall    : 0.2063 (20.63%)
F1-Score  : 0.3229 (32.29%)

================================================
SCIKIT-LEARN VERIFICATION
================================================
Accuracy  : 0.9244
Precision : 0.7431986809563067
Recall    : 0.2062929061784897
F1-Score  : 0.322944653412144

================================================
INTERPRETATION
================================================
Accuracy  : Overall percentage of correct predictions.
Precision : Percentage of predicted frauds that are actually fraud.
Recall    : Percentage of actual frauds detected by the model.
F1-Score  : Balance between Precision and Recall.

Important:
Recall is especially important in fraud detection
because False Negatives represent fraudulent
transactions that were incorrectly classified as legitimate.
```

# Task 10: Plot ROC Curve, Calculate AUC and Interpret Results

Calculate the **False Positive Rate (FPR)** and **True Positive Rate (TPR)** for different classification thresholds.

The False Positive Rate is calculated as:

$$ FPR = \frac{FP}{FP+TN} $$
The True Positive Rate is equivalent to Recall and is calculated as:

$$ TPR = \frac{TP}{TP+FN} $$
Plot the **Receiver Operating Characteristic (ROC) Curve)** by plotting FPR on the (x)-axis and TPR on the (y)-axis for different classification thresholds.

Calculate the **Area Under the ROC Curve (AUC)** to measure the ability of the Logistic Regression model to distinguish between fraudulent and legitimate transactions.

Prepare a final performance summary containing:

- Accuracy
- Precision
- Recall
- F1-Score
- ROC-AUC
- Confusion Matrix

Interpret the model performance and discuss the trade-off between **False Positives and False Negatives**.

A higher ROC-AUC value indicates better ability of the model to distinguish between fraudulent and legitimate transactions.

Explain whether the trained Logistic Regression model is suitable as a **baseline model for credit card fraud detection**.

The objective of this task is to evaluate the overall classification performance of the Logistic Regression model using the ROC Curve and AUC and to understand the trade-off between detecting fraudulent transactions and incorrectly flagging legitimate transactions.

In [11…

```
# ============================================================
# TASK 10: ROC CURVE, AUC AND FINAL PERFORMANCE SUMMARY
# ============================================================

import numpy as np
import matplotlib.pyplot as plt

from sklearn.metrics import (
    roc_curve,
    roc_auc_score,
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)


# ------------------------------------------------------------
# 1. Generate Fraud Probabilities
# ------------------------------------------------------------

# Calculate probability of fraud for test data
test_probability = sigmoid(
    np.dot(X_test_np, w) + b
)

# Convert probability to class
# Threshold = 0.5
test_prediction = (
    test_probability >= 0.5
).astype(int)


# ------------------------------------------------------------
# 2. Calculate ROC Curve
# ------------------------------------------------------------

fpr, tpr, thresholds = roc_curve(
    y_test_np,
    test_probability
)


# ------------------------------------------------------------
# 3. Calculate ROC-AUC
# ------------------------------------------------------------

roc_auc = roc_auc_score(
    y_test_np,
    test_probability
)
```

```
print("================================================")
print("ROC-AUC RESULTS")
print("================================================")

print("ROC-AUC:", roc_auc)


# ------------------------------------------------------------
# 4. Plot ROC Curve
# ------------------------------------------------------------

plt.figure(figsize=(8, 6))

plt.plot(
    fpr,
    tpr,
    label="Logistic Regression (AUC = {:.4f})".format(
        roc_auc
    )
)

# Random classifier reference line
plt.plot(
    [0, 1],
    [0, 1],
    linestyle="--",
    label="Random Classifier"
)

plt.xlabel("False Positive Rate (FPR)")
plt.ylabel("True Positive Rate (TPR)")
plt.title("ROC Curve - Logistic Regression")

plt.legend()
plt.grid(True)

plt.show()


# ------------------------------------------------------------
# 5. Calculate Confusion Matrix
# ------------------------------------------------------------

cm = confusion_matrix(
    y_test_np,
    test_prediction
)

TN = cm[0, 0]
FP = cm[0, 1]
FN = cm[1, 0]
TP = cm[1, 1]
```

```
# ------------------------------------------------------------
# 6. Calculate Performance Metrics
# ------------------------------------------------------------

accuracy = accuracy_score(
    y_test_np,
    test_prediction
)

precision = precision_score(
    y_test_np,
    test_prediction,
    zero_division=0
)

recall = recall_score(
    y_test_np,
    test_prediction,
    zero_division=0
)

f1 = f1_score(
    y_test_np,
    test_prediction,
    zero_division=0
)


# ------------------------------------------------------------
# 7. Display Confusion Matrix
# ------------------------------------------------------------

print("\n================================================")
print("CONFUSION MATRIX")
print("================================================")

print("\n                 Predicted")
print("                Legitimate    Fraud")

print(
    "Actual Legitimate     ",
    TN,
    "          ",
    FP
)

print(
    "Actual Fraudulent     ",
    FN,
    "          ",
    TP
)
```

```python
# ------------------------------------------------------------
# 8. Display Final Performance Summary
# ------------------------------------------------------------

print("\n================================================")
print("FINAL PERFORMANCE SUMMARY")
print("================================================")

print(
    "Accuracy  : {:.4f} ({:.2f}%)".format(
        accuracy,
        accuracy * 100
    )
)

print(
    "Precision : {:.4f} ({:.2f}%)".format(
        precision,
        precision * 100
    )
)

print(
    "Recall    : {:.4f} ({:.2f}%)".format(
        recall,
        recall * 100
    )
)

print(
    "F1-Score  : {:.4f} ({:.2f}%)".format(
        f1,
        f1 * 100
    )
)

print(
    "ROC-AUC   : {:.4f}".format(
        roc_auc
    )
)

print("\nConfusion Matrix:")
print(cm)


# ------------------------------------------------------------
# 9. Display FPR and TPR
# ------------------------------------------------------------

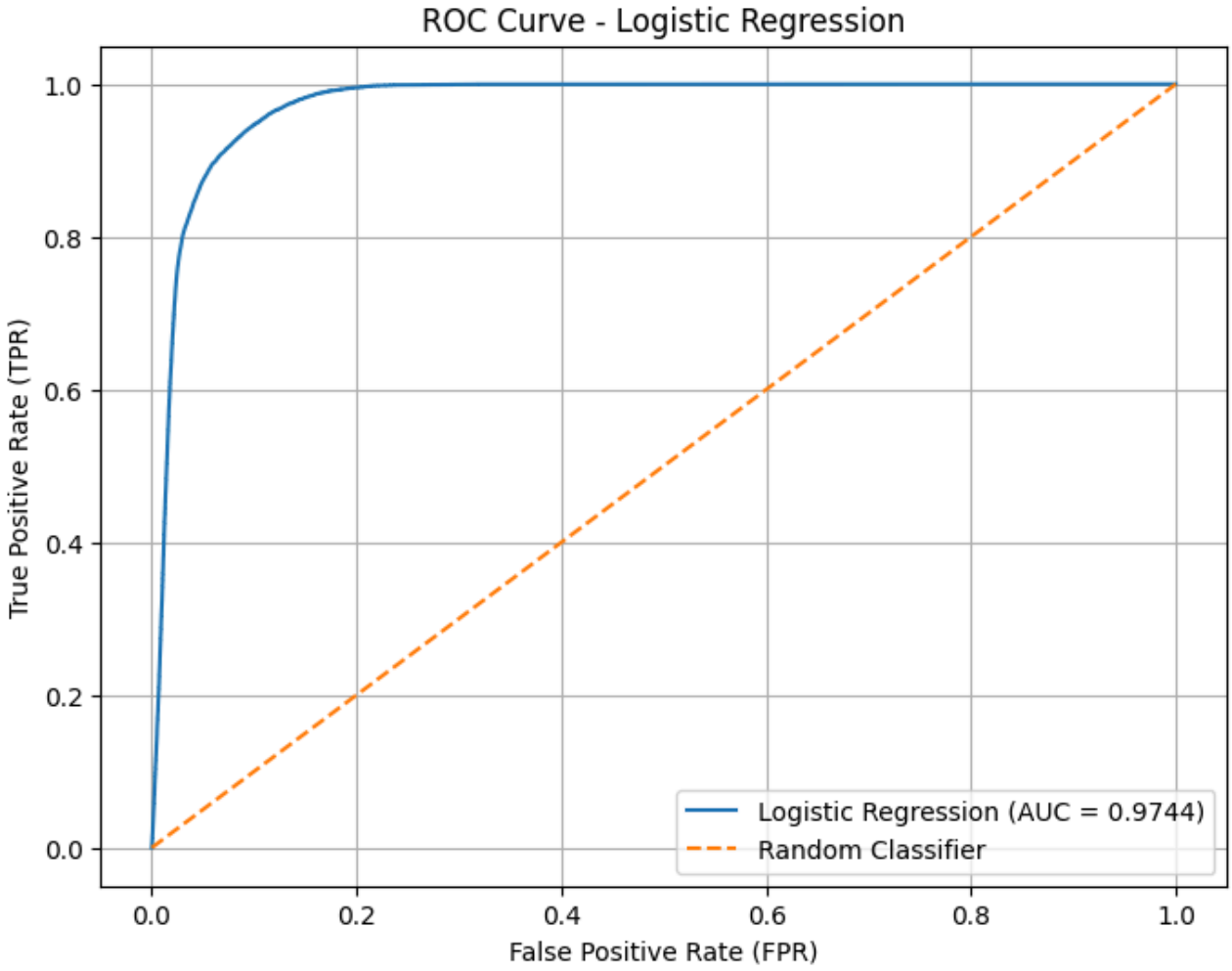
print("\n================================================")
print("FPR AND TPR")
print("================================================")
```

```
print("False Positive Rate (FPR):")
print(fpr)

print("\nTrue Positive Rate (TPR):")
print(tpr)

print("\nClassification Thresholds:")
print(thresholds)
```

```
==================================================
ROC-AUC RESULTS
==================================================
ROC-AUC: 0.9743585715944235
```



```
================================================
CONFUSION MATRIX
================================================

                 Predicted
                 Legitimate    Fraud
Actual Legitimate      181274            1246
Actual Fraudulent      13874            3606

================================================
FINAL PERFORMANCE SUMMARY
================================================
Accuracy  : 0.9244 (92.44%)
Precision : 0.7432 (74.32%)
Recall    : 0.2063 (20.63%)
F1-Score  : 0.3229 (32.29%)
ROC-AUC   : 0.9744

Confusion Matrix:
[[181274   1246]
 [ 13874   3606]]

================================================
FPR AND TPR
================================================
False Positive Rate (FPR):
[0.00000000e+00 5.47885163e-06 5.47885163e-06 ... 6.16359851e-01
 6.16359851e-01 1.00000000e+00]

True Positive Rate (TPR):
[0.00000000e+00 0.00000000e+00 2.28832952e-04 ... 9.99942792e-01
 1.00000000e+00 1.00000000e+00]

Classification Thresholds:
[       inf 1.         0.99999998 ... 0.11933704 0.11933665 0.03698467]
```

# Conclusion

The experiment successfully demonstrated the implementation of **Logistic Regression for credit card fraud detection**. The complete workflow included data preprocessing, exploratory data analysis, feature scaling, class-imbalance handling, dataset splitting, Sigmoid Function implementation, Binary Cross-Entropy Cost Function, and Gradient Descent optimization.

The trained Logistic Regression model was evaluated using the **Confusion Matrix, Accuracy, Precision, Recall, F1-Score, ROC Curve, and ROC-AUC**. The results demonstrate that accuracy alone is not an adequate performance measure for highly imbalanced fraud detection datasets.

Particular importance was given to **Recall**, because a False Negative represents a fraudulent transaction that is incorrectly classified as legitimate. A suitable fraud detection model should therefore achieve satisfactory Recall while maintaining an acceptable level of False Positives.

Overall, Logistic Regression provides an **interpretable and effective baseline model** for credit card fraud detection. The experiment also demonstrates the importance of selecting appropriate evaluation metrics and handling class imbalance when developing machine learning models for fraud detection.